Salesforce Programmatic Model 1

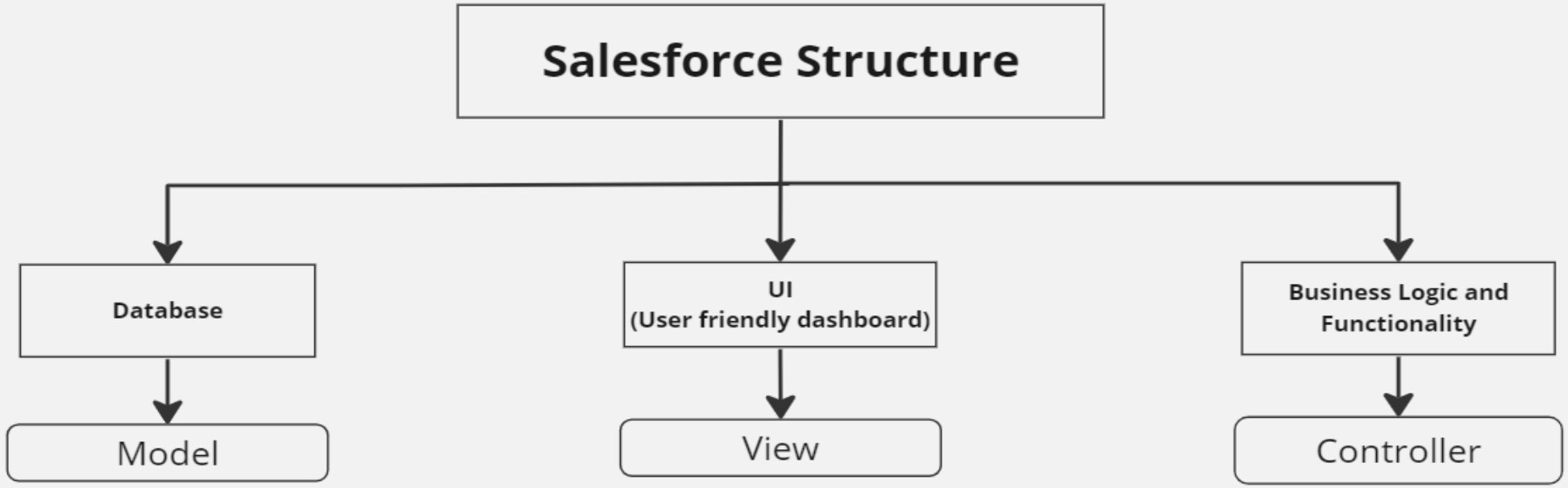
Agenda

1. Why to Code ?
2. Introduction to Apex Language
3. Architecture of language(How does apex works?)
4. Language constructs
5. Apex naming conventions
6. Data Types
7. Debugging

Why to Code ?

1. To create **custom business logic**, **validations, and functionalities** that go **beyond the standard capabilities of Salesforce's point-and-click tools**.
2. To provide **more flexibility and develop Customized applications** through custom logic
3. To meet **Personalized requirements** of client
4. To **Communicate/collaborate** with **external systems / integrations**
5. For **handling Bulk data** (more than 50,000 records)
6. To handle operations through **asynchronous process**
7. Complex **business processes** that are unsupported by configuration
8. To respond to events (such as record updates, inserts, deletes, undeletes) and perform **custom actions or updates**

Introduction to Apex Language



Declarative	Programmatic
1. Standard Objects 2. Custom Objects 3. Fields 4. Records	1. sObjects(Generic) 2. Wrapper / Inner Classes 3. Variables (Primitives, Non Primitives) 4. Instance

Declarative	Programmatic
1. Page Layouts 2. Lightning Home Pages 3. Lightning App page 4. Lightning Record Page	1. Visualforce pages 2. Lightning Components (Aura) 3. Lightning Web Components (LWC)

Declarative	Programmatic
1. Workflows 2. Process Builders 3. Flows 4. Assignment Rules 5. Auto Response Rules 6. Validation Rules	1. Apex Classes 2. Apex Triggers 3. Batch Apex 4. Queueable Apex 5. Schedule Apex 6. Asynchronous Apex

Introduction to Apex Language

1. Apex is a strongly typed, object-oriented programming language that allows developers to execute flow and transaction control statements on Salesforce servers.
2. Apex is a development platform for building software as a service (SaaS) applications on top of Salesforce.com's customer relationship management (CRM) functionality.
3. Uses syntax that looks like Java and acts like database stored procedures.
4. Allows developers to add business logic.
5. Apex applications are usually hosted and run directly from Salesforce.com's servers.

Apex is :

- Rigorous
  - Strongly-typed language that uses direct references to schema objects such as object and field names
  - Fails quickly at compile time if any references are invalid

- Hosted
  - Apex is interpreted, executed, and controlled entirely by the Force.com platform.
  - Apex allows developers to create business logic, automation, and custom processes within the Salesforce ecosystem

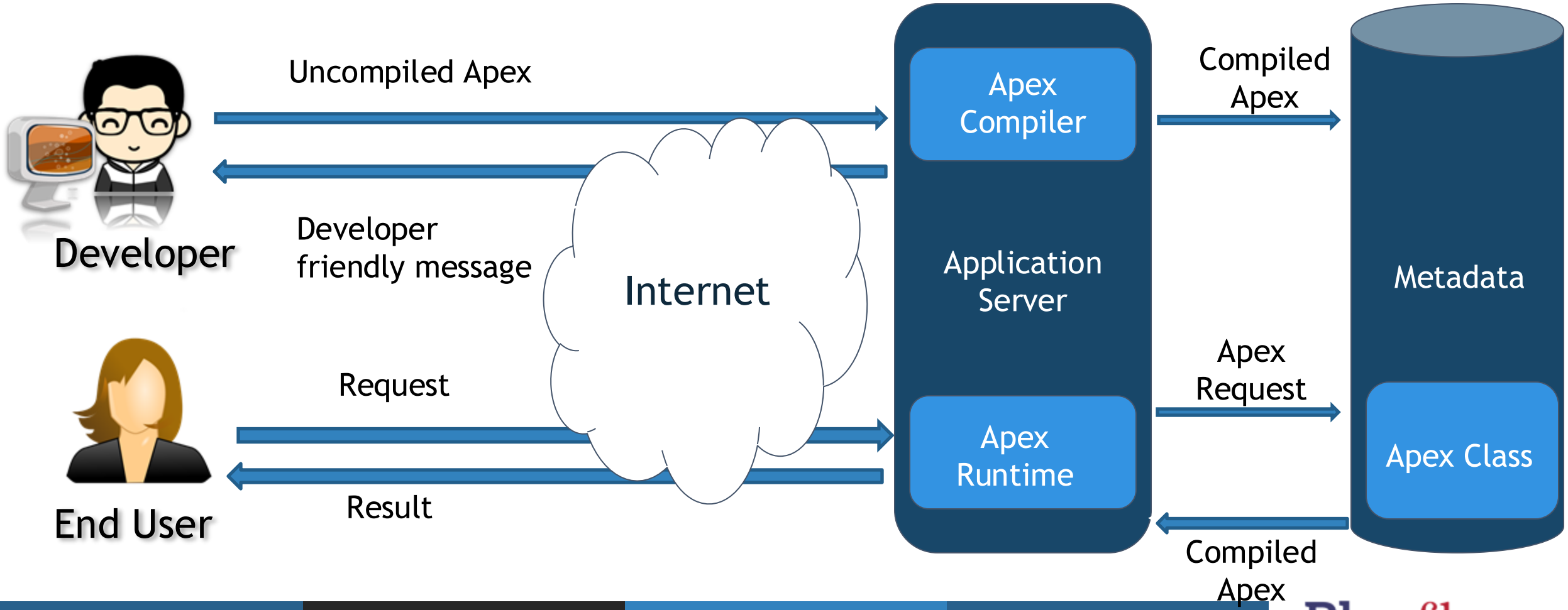
Apex is:

Multitenant Aware： This allows multiple organizations to use the same infrastructure and resources while ensuring fair resource allocation, data isolation, security, and scalability.

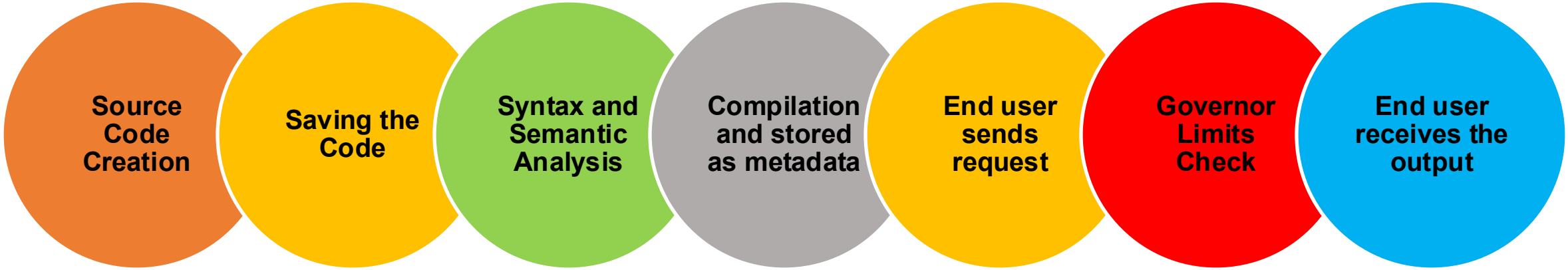
Automatically Upgradeable

Easy to Test

How does Apex Works ?



How does Apex Works ?



How to Invoke Apex?

To Attach Business logic to an Application

On UI Events

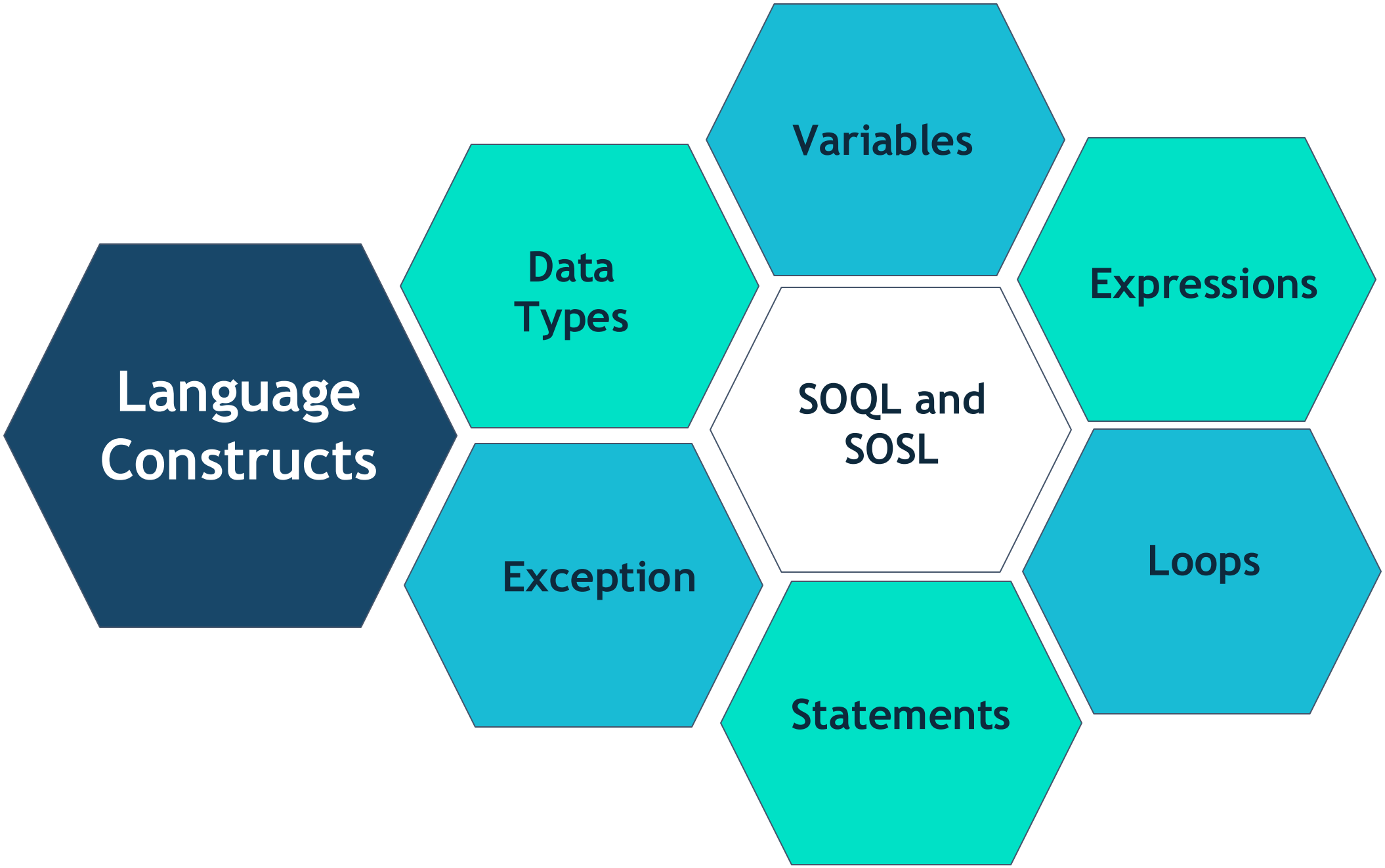
For example: When user clicks on a button from UI, gives input

On System Events

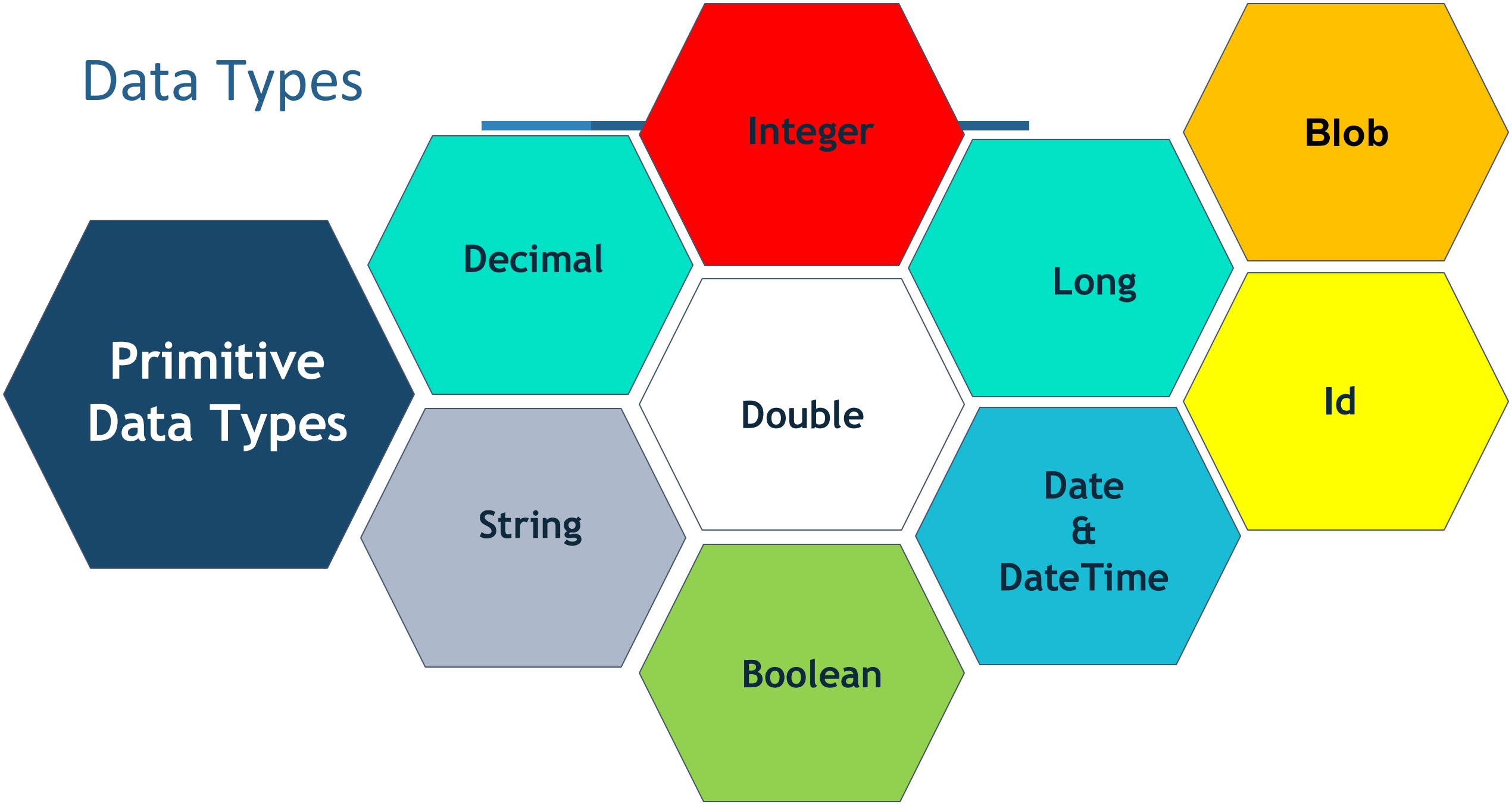
For example: After a specific time period, a process must run

On Database Events

For example: On creation, update or deletion of a record


Language Constructs
Data Types
Variables
Expressions
SOQL and SOSL
Exception
Loops
Statements

Data Types

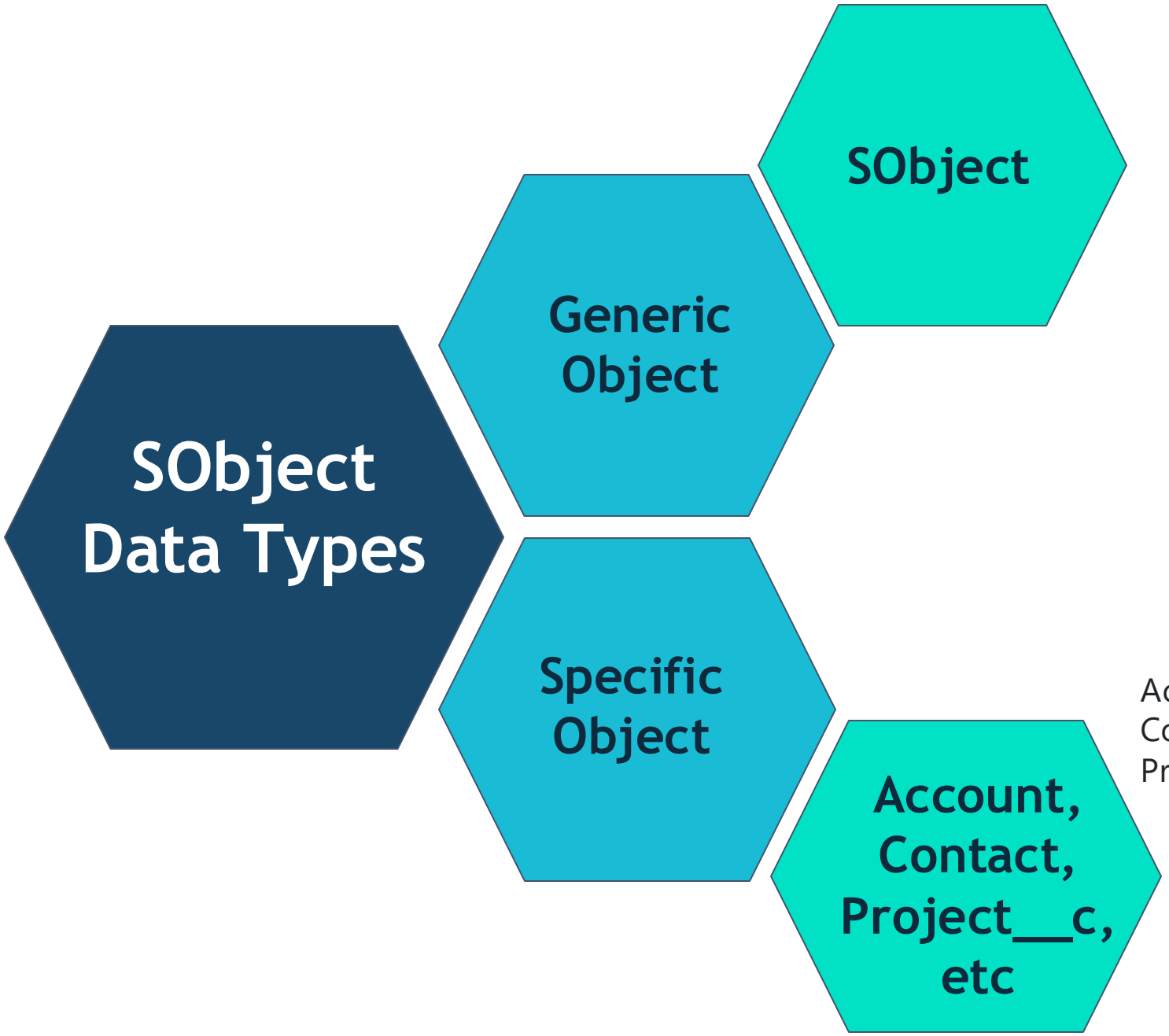

Primitive Data Types
Integer
Blob
Decimal
Long
Double
Id
String
Date & DateTime
Boolean

Blob Data Type

- In Salesforce Apex, the Blob data type represents binary data, which can include a variety of binary content such as images, audio files, documents, and more. The term "Blob" stands for Binary Large Object. The Blob data type is used to store, manipulate, and process binary data within Apex code.

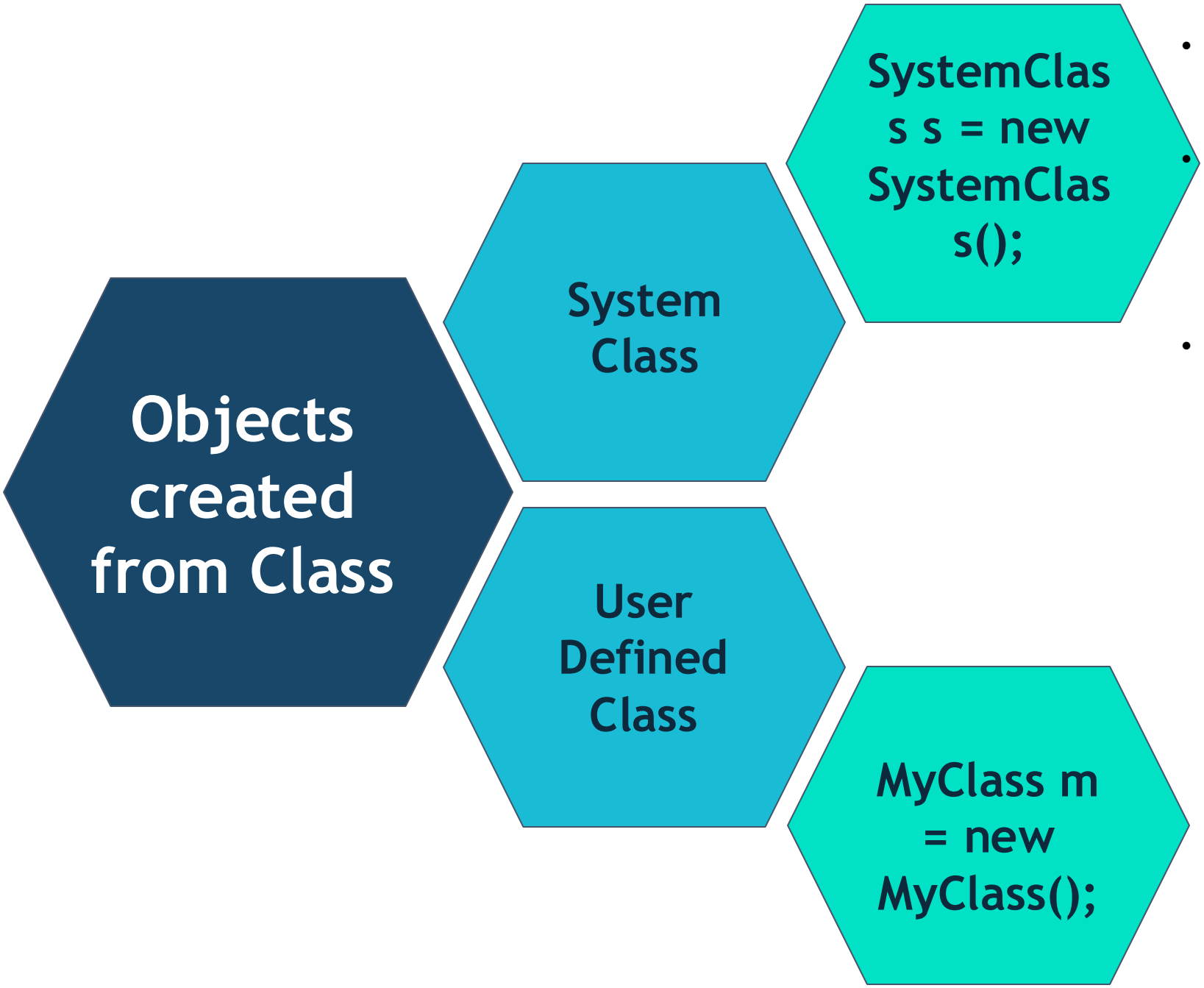
- **Binary Data:** A Blob represents a sequence of binary bytes. It can hold any type of binary data, such as images, PDFs, ZIP files, and so on.

```
// Encoding a string to a Blob
String originalString = 'Hello, world!';
Blob encodedBlob = Blob.valueOf(originalString);
```




```
sObject s1 = new Account(Name = BFL);
sObject s2 = new Contact(lastName = ‘Jacobs’);
sObject s3 = new Project__c(Name = 'Adidas');
```

```
Account accountObj = new Account(Name = BFL);
Contact contactObj = new Contact(lastName = ‘Jacobs’);
Project__c projectObj = new Project__c(Name = 'Adidas');
```



- The "**System class**" refers to a set of **predefined classes provided by the Salesforce platform**.
- These classes are part of the Salesforce runtime environment and offer a wide range of functionalities to interact with the platform, perform operations on data, manage security, and more.
- Examples: `System.debug`, `System.assert`, `System.Math etc.`

•A **"user-defined class"** is a class that you create yourself to encapsulate custom logic, data, and behavior within your Apex code.
•These classes are defined by developers to meet specific requirements or implement custom business processes.
•User-defined classes can have properties, methods, constructors, and other members that you define.
•They are used to organize code, improve code reusability, and implement custom business logic.

Collections

List/Array

For example: List<Account> accountList = new List<Account>();

Set

For example: Set<Id> accountIdSet = new Set<Id>();

Map

For example: Map<Id, Account> accountIdVSAccountMap = new Map<Id, Account>();

Variables

For example: Integer quantity;

Expressions

For example: 1 + 1

Exceptions

For example: DML exception

Conditional (If-Else) Statements

For example:

```
if(condition) {
} else {
}
```

Switch Statements

For example:

```
switch on expression {
  when value1 { // when block 1
  }
  when value2 { // when block 2
  }
}
```

Loops

For Loops

Traditional For Loops

```
for (init_stmt; exit_condition; increment_stmt)
{
    code_block
}

for (Integer i = 0, j = 0; i < 10; i++) {
    System.debug(i+1);
}
```

List or Set Iteration for Loops

```
for (variable : list_or_set) {
    code_block
}

Integer[] myInts = new Integer[]{1, 2, 3, 4};
//List<Integer> myInts = new List<Integer>{1, 2,
3, 4};

for (Integer i : myInts) {
    System.debug(i);
}
```

Apex Class Syntax

```
private | public | global
[virtual | abstract | with sharing | without sharing | inherited sharing]
class ClassName
[implements InterfaceNameList]
 [extends ClassName]
{
// The body of the class
}
```

Apex Naming Conventions

- **Class Name and Helper Class**:
  - Use PascalCase (capitalize the first letter of each word) for class names.
  - Choose descriptive and meaningful names that convey the purpose of the class.
  - <FunctionalEntity> & <ClassName>Helper
  - Ex: CalculatorProcessor, CalculatorProcessorHelper, ProjectController, ProjectControllerHelper

```
public class AccountProcessor {
    // Class members and methods
}
```

Apex Naming Conventions

- **Method/Function Names:**
  - Use camelCase (capitalize the first letter of each word except the first) for method names.
  - Use descriptive names that indicate the purpose of the method.

```
public void calculateTotalAmount() {
    // Method logic
}
```

Apex Naming Conventions

- **Variable and Property Names:**
  - Use camelCase for variable and property names.
  - Choose meaningful names that reflect the purpose of the variable or property.

```
Integer itemCount = 10;
String customerName = 'John Doe';
```

Apex Naming Conventions

- **Constant Names:**
  - Use uppercase letters with underscores for constants.
  - Constants are often declared as static final variables.

```
public static final Integer MAX_RETRIES = 3;
```

- **Trigger and Handler:** <ObjectName>Trigger, <ObjectName>TriggerHandler.
  Example : AccountTrigger, AccountTriggerHandler, CaseTrigger, CaseTriggerHelper

Apex Naming Conventions

- **UI (Page, Lighting Component etc.):** Camel Case with first letter capital.
- **Other Development Components:** Camel Case with first letter capital.
- **Collection Name:** The collection variable name should always end with its type

  Map, List, Set
  - Ex. : contactList, idVsContactMap, accountIdVSContactsMap, accountIdSet.

```apex
/**
 * @Description: This class contains various methods to handle various calculations for the provided inputs
 * @Author: Sachin Vispute
 * Version       Date            Author            Modification
 * 1.0         07-03-2022     Sachin Vispute       Intial Draft
 * 2.0         09-04-2022     Sachin Vispute       Final Release
 */
public with sharing class CalculatorProcessor {
    public static final Double PI_VALUE = 3.14; // Storing Value of 'Pi' into constant

    /**
     * AB-1234
     * @description     :   This method is used to perform summation of provided numbers
     * @author          :   Sachin V
     * @param NumberOne :   Number provided while invoking this method
     * @param NumberTwo :   Number provided while invoking this method
     * @return sum      :   Result of summation of two numbers provided
     */
    public static Decimal doCalculation(Decimal numberOne, Decimal numberTwo) {
        if (numberOne != null && numberTwo != null) {
            return numberOne + numberTwo;
        } else {
            return null;
        }
    }
```

Debugging

Apex provides debugging support. You can debug your Apex code using the following:

- Debug Logs
- System.debug
- Developer Console

Additional Resources

- Apex Developer Guide
  https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_dev_guide.htm
- Trailhead
- Blogs
- Youtube

THANK YOU

Blueflame

Contact Us:
www.theblueflamelabs.com
fireitup@theblueflamelabs.com
+1 (626) 427-8157